# DOKTORAK

## Doctor Appointment Management System

### Software Engineering Project Documentation

# 1. Project Planning & Management

## 1.1 Project Proposal

### Project Title

**Doktorak – Doctor Appointment Management System**

### Project Overview

Doktorak is a web-based Doctor Appointment Management System designed to simplify the process of scheduling and managing medical appointments. The system provides a centralized platform that connects patients, doctors, and administrators, enabling efficient appointment booking, schedule management, and healthcare service administration.

Patients can register, search for doctors based on specialization, location, or gender, view doctor profiles, book appointments, manage their profiles, and submit reviews after completed appointments. Doctors can manage their professional profiles, create and update their schedules, respond to appointment requests, and monitor appointment statistics. Administrators are responsible for managing doctors, clinics, specializations, patients, and monitoring the overall operation of the system.

The system aims to replace traditional manual appointment booking methods with a secure, user-friendly, and efficient digital solution.

---

### Problem Statement

Many healthcare facilities still depend on manual appointment booking through phone calls or in-person visits. These traditional methods introduce several challenges, including:

- Long waiting times for patients.
- Appointment conflicts and double bookings.
- Difficulty in finding doctors based on specialization or location.
- Lack of centralized patient and doctor information.
- Time-consuming schedule management for doctors.
- Limited monitoring and reporting capabilities for administrators.

These issues reduce healthcare efficiency and negatively impact both patients and healthcare providers.

---

## Proposed Solution

Doktorak addresses these challenges by providing an integrated web application that allows:

- Patients to register, manage their profiles, browse doctors, and book appointments online.
- Doctors to manage their schedules and appointment requests.
- Administrators to manage doctors, clinics, specializations, and monitor system activities.
- Secure authentication and role-based authorization to protect user data.
- Centralized management of appointments and healthcare resources.

---

## Project Objectives

The primary objectives of Doktorak are:

- Develop a secure online doctor appointment management system.
- Simplify appointment booking for patients.
- Enable doctors to efficiently manage their schedules.
- Reduce appointment conflicts and manual administrative work.
- Provide administrators with centralized management tools.
- Improve communication between patients and healthcare providers.
- Enhance the overall healthcare service experience.

---

## Project Scope

The project includes three main user roles:

**Patient**

- Register and log in.

- Manage personal profile.
- Upload profile picture.
- Browse doctors.
- Search and filter doctors.
- View doctor profiles.
- Book appointments.
- Cancel appointments.
- View upcoming appointments.
- View appointment history.
- Submit doctor reviews.

**Doctor**

- Log in.
- Manage personal profile.
- Upload profile picture.
- Create, edit, and delete schedules.
- View appointments.
- Accept or reject appointment requests.
- Mark appointments as completed.
- View patient information.
- View appointment statistics.

**Administrator**

- Monitor dashboard statistics.
- Add doctors.
- Activate or deactivate doctors.
- View registered patients.
- Manage clinics.
- Manage medical specializations.
- Monitor all appointments.

---

## Expected Deliverables

The project deliverables include:

- Functional Doctor Appointment Management System.
- Source code hosted on GitHub.
- Database implementation using SQL Server.
- Software documentation.
- UML diagrams.
- Deployment on MonsterASP.NET.

- User manual.
- Project presentation.

---

# 1.2 Project Plan

## Development Methodology

The Doktorak project follows the **Agile Software Development Methodology**, where development is divided into multiple iterations. Each iteration focuses on implementing a specific set of features, followed by testing and evaluation before moving to the next phase.

---

## Project Timeline

| Phase | Main Activities | Deliverable |
|---|---|---|
| Project Planning | Define objectives, scope, and team responsibilities | Project Proposal |
| Requirements Gathering | Collect functional and non-functional requirements | Software Requirements Specification |
| System Analysis & Design | Create UML diagrams, database design, and architecture | System Design Document |
| Implementation | Develop backend, frontend, and database | Working Application |
| Testing | Functional testing, validation, and bug fixing | Tested System |
| Deployment | Publish application and database | Live System |
| Documentation | Prepare technical documentation and user manual | Final Documentation |
| Presentation | Demonstrate the system | Final Presentation |

---

## Milestones

| Milestone | Description |
|---|---|

| | |
|---|---|
| M1 | Complete project planning |
| M2 | Finish requirements gathering |
| M3 | Complete system analysis and design |
| M4 | Finish database implementation |
| M5 | Complete backend development |
| M6 | Complete frontend implementation |
| M7 | Complete system testing |
| M8 | Deploy the application |
| M9 | Submit documentation and presentation |

---

## Resources

### Software

- Visual Studio 2022
- SQL Server
- SQL Server Management Studio (SSMS)
- Git
- GitHub
- Draw.io / Lucidchart (for UML diagrams)
- MonsterASP.NET (Hosting)

### Technologies

- ASP.NET Core MVC
- Entity Framework Core
- SQL Server
- HTML5
- CSS3
- Bootstrap 5
- JavaScript

---

# 1.3 Task Assignment & Roles

The project was developed collaboratively by a software development team. Responsibilities were distributed to ensure efficient development and proper coordination among members.

| Role | Responsibilities |
|---|---|
| Team Leader | Project planning, task coordination, GitHub repository management, documentation review, system integration, and deployment. |
| Backend Developers | Implement business logic, authentication, authorization, appointment management, schedule management, and database interaction. |
| Frontend Developers | Design and implement user interfaces using HTML, CSS, Bootstrap, and JavaScript while ensuring responsive and user-friendly pages. |
| Database Designer | Design the database schema, relationships, constraints, and optimize SQL queries. |

---

### Team Collaboration

The team used GitHub for version control and collaborative development.

The collaboration workflow included:

- Creating feature branches for new functionality.
- Committing changes with meaningful commit messages.
- Reviewing code before merging into the main branch.
- Resolving merge conflicts collaboratively.
- Tracking project progress through GitHub.

---

## 1.4 Risk Assessment & Mitigation Plan

| Risk | Probability | Impact | Mitigation Strategy |
|---|---|---|---|
| Requirement changes during development | Medium | High | Conduct regular meetings with the supervisor and update documentation accordingly. |
| Team member availability | Medium | Medium | Distribute tasks evenly and maintain continuous communication among team members. |

| | | | |
|---|---|---|---|
| Database design issues | Low | High | Review database design before implementation and perform frequent testing. |
| Integration conflicts between modules | Medium | High | Use GitHub branching strategy and perform regular integration testing. |
| Deployment failures | Medium | Medium | Test deployment in a staging environment before publishing the final system. |
| Data loss | Low | High | Maintain regular backups of the database and source code repository. |
| Security vulnerabilities | Medium | High | Implement authentication, authorization, password hashing, and input validation. |
| Schedule delays | Medium | Medium | Monitor project milestones and adjust task priorities when necessary. |

---

## 1.5 KPIs (Key Performance Indicators)

The success of the Doktorak system can be evaluated using the following Key Performance Indicators:

| KPI | Description | Target |
|---|---|---|
| System Availability | Percentage of time the system remains operational. | ≥ 99% |
| Login Success Rate | Percentage of successful user login attempts. | ≥ 98% |
| Average Page Response Time | Average time required to load system pages. | Less than 3 seconds |
| Appointment Booking Success Rate | Percentage of appointments booked successfully without conflicts. | ≥ 95% |
| Appointment Conflict Rate | Percentage of double-booked or conflicting appointments. | 0% |
| Doctor Search Performance | Time required to retrieve doctor search results. | Less than 2 seconds |

| | | |
|---|---|---|
| User Satisfaction | User feedback collected through surveys and reviews. | ≥ 90% satisfaction |
| Database Integrity | Accuracy and consistency of stored data. | 100% |
| System Security | Number of successful unauthorized access attempts. | 0 incidents |
| Deployment Success | Successful deployment without critical issues. | 100% successful deployment |

---

# 2. Literature Review

## 2.1 Literature Review

### Introduction

Healthcare appointment management has evolved significantly with the advancement of information technology. Traditional appointment booking methods, such as phone calls and walk-in scheduling, often lead to long waiting times, scheduling conflicts, and inefficient communication between patients and healthcare providers. To overcome these challenges, many healthcare organizations have adopted online appointment management systems that automate scheduling and improve accessibility.

Doctor appointment systems aim to simplify the interaction between patients, doctors, and healthcare administrators by providing secure online services such as appointment booking, schedule management, patient records, and doctor discovery. These systems improve operational efficiency while enhancing the overall patient experience.

---

### Existing Systems

Several healthcare appointment systems currently exist, each offering different features and capabilities.

#### 1. Vezeeta

Vezeeta is one of the most popular healthcare platforms in the Middle East. It allows patients to search for doctors by specialty, location, and insurance provider. Patients can view doctor profiles, read reviews, and book appointments online.

**Strengths**

- Large network of doctors.
- Easy appointment booking.
- Doctor ratings and reviews.
- Mobile application support.

**Limitations**

- Limited administrative customization.
- Some premium features require payment.
- Limited flexibility for healthcare organizations that require customized workflows.

---

**2. Practo**

Practo is an international healthcare platform providing appointment booking, telemedicine, and digital healthcare services.

**Strengths**

- Online consultation.
- Electronic medical records.
- Medicine ordering.
- Appointment reminders.

**Limitations**

- Complex interface for new users.
- Many advanced services require subscription plans.
- Some features are unavailable in certain regions.

---

**3. Hospital Management Systems**

Many hospitals use integrated Hospital Management Systems (HMS) that include appointment scheduling as one component among patient records, billing, pharmacy, and laboratory management.

**Strengths**

- Complete healthcare management.
- Centralized patient information.
- Integration with hospital departments.

**Limitations**

- Expensive implementation.
- Complex deployment.
- Requires extensive training.

---

## Comparative Analysis

| Feature | Traditional Booking | Vezeeta | Practo | Doktorak |
|---|---|---|---|---|
| Online Registration | ✖ | ✔ | ✔ | ✔ |
| Doctor Search | ✖ | ✔ | ✔ | ✔ |
| Filter by Specialization | ✖ | ✔ | ✔ | ✔ |
| Filter by Location | ✖ | ✔ | ✔ | ✔ |
| Appointment Booking | ✖ | ✔ | ✔ | ✔ |
| Appointment Cancellation | ✖ | ✔ | ✔ | ✔ |
| Doctor Schedule Management | ✖ | Limited | ✔ | ✔ |
| Administrator Dashboard | ✖ | Limited | Limited | ✔ |
| Clinic Management | ✖ | ✖ | Limited | ✔ |
| Specialization Management | ✖ | ✖ | Limited | ✔ |
| Appointment Monitoring | ✖ | ✖ | Limited | ✔ |
| Patient Reviews | ✖ | ✔ | ✔ | ✔ |

---

## Motivation for Developing Doktorak

Although many appointment management systems already exist, they are often commercial platforms designed for large healthcare providers or subscription-based services. The Doktorak project was developed to provide a simplified and customizable solution that focuses on the essential needs of appointment management while demonstrating software engineering principles.

The system emphasizes:

- Secure authentication and authorization.
- Efficient appointment scheduling.
- Doctor schedule management.
- Administrative control over clinics and specializations.
- User-friendly interfaces.
- Centralized healthcare information.

---

### Conclusion

The literature review demonstrates that online appointment systems significantly improve healthcare service efficiency by reducing manual work, minimizing scheduling conflicts, and enhancing communication between patients and doctors. Inspired by the strengths of existing platforms while addressing their limitations, Doktorak provides a comprehensive web-based solution tailored to support patients, doctors, and administrators within a single integrated system.

---

# 2.2 Feedback & Evaluation

Throughout the development process, continuous feedback was collected from the project supervisor and team members to evaluate the quality and functionality of the system.

### Supervisor Feedback

The supervisor emphasized the importance of:

- Following software engineering best practices.
- Maintaining a clear layered architecture.
- Applying proper authentication and authorization.
- Designing a normalized database.
- Producing comprehensive project documentation.
- Ensuring consistency across user interfaces.

---

### Team Evaluation

The development team regularly reviewed completed features to ensure they met the project requirements.

Evaluation focused on:

- Correct implementation of functional requirements.
- User interface consistency.
- Database integrity.
- Code readability and maintainability.
- Integration between system modules.
- Error handling and validation.

---

## User Perspective

The system was evaluated based on the expected experience of its three primary user groups.

**Patients**

Patients should be able to:

- Register and log in easily.
- Find suitable doctors quickly.
- Book appointments without conflicts.
- Manage their appointments efficiently.

**Doctors**

Doctors should be able to:

- Manage schedules easily.
- Respond to appointment requests.
- Access patient information when needed.

**Administrators**

Administrators should be able to:

- Manage healthcare resources.
- Monitor appointments.
- Manage clinics and specializations.
- View overall system statistics.

---

## Evaluation Summary

The implemented system successfully satisfies the defined functional requirements by providing secure user authentication, efficient appointment scheduling, comprehensive doctor management, and centralized administration. The modular architecture also supports future expansion and maintenance.

---

# 2.3 Suggested Improvements

Although Doktorak provides all core functionalities required for a doctor appointment management system, several enhancements can further improve the platform in future versions.

### Feature Enhancements

- Online payment integration.
- Email appointment confirmations.
- SMS appointment reminders.
- Push notifications.
- Telemedicine video consultations.
- AI-based doctor recommendations.
- Multi-language support (Arabic and English).
- Mobile applications for Android and iOS.
- Calendar synchronization with Google Calendar and Outlook.

---

### System Improvements

- Improve search performance for large datasets.
- Implement caching mechanisms.
- Add advanced analytics dashboards.
- Enhance accessibility for users with disabilities.
- Introduce audit logging for administrative actions.

---

### Security Improvements

- Two-factor authentication (2FA).
- Password recovery via email.
- Login attempt monitoring.
- Activity logs.
- Enhanced data encryption.

- Automatic session timeout after inactivity.

---

### Healthcare Enhancements

- Electronic medical records (EMR).
- Digital prescriptions.
- Laboratory test integration.
- Medical history tracking.
- Insurance management.
- Doctor availability forecasting.

---

# 2.4 Final Grading Criteria

The Doktorak project was developed to satisfy the evaluation criteria commonly used in software engineering projects. The following aspects were considered throughout development.

| Evaluation Criterion | Description |
|---|---|
| Requirements Analysis | Functional and non-functional requirements were identified, documented, and implemented. |
| System Design | UML diagrams, database design, and software architecture were prepared according to software engineering principles. |
| Database Design | A normalized SQL Server database was designed with appropriate relationships and constraints. |
| Implementation | The system was developed using ASP.NET Core MVC, Entity Framework Core, Bootstrap, and SQL Server. |
| Code Quality | Modular architecture, reusable components, and coding standards were followed to improve maintainability. |
| User Interface | Responsive and user-friendly interfaces were designed for patients, doctors, and administrators. |
| Security | Authentication, authorization, password hashing, and input validation were implemented to secure the application. |

| | |
|---|---|
| Testing | Functional testing and validation were performed to verify that all features operate correctly. |
| Deployment | The application was successfully deployed to MonsterASP.NET with a SQL Server database. |
| Documentation | Comprehensive documentation, including requirements, design, implementation, and testing, was prepared to support the project. |

---

# 3. Requirements Gathering

## 3.1 Stakeholder Analysis

### Introduction

Stakeholder analysis is the process of identifying all individuals and groups that interact with the system and understanding their needs and responsibilities. For the Doktorak Doctor Appointment Management System, stakeholders are categorized into primary stakeholders, who directly use the system, and secondary stakeholders, who benefit from or maintain the system.

---

### Stakeholder Identification

| Stakeholder | Description | Responsibilities |
|---|---|---|
| Patient | A registered user seeking medical consultation. | Register an account, search for doctors, book appointments, manage profile, view appointment history, submit reviews. |
| Doctor | A healthcare professional providing medical services. | Manage profile, create and manage schedules, respond to appointment requests, complete appointments, view patient information. |
| Administrator | System manager responsible for maintaining the platform. | Manage doctors, clinics, specializations, patients, appointments, and monitor system statistics. |

| | | |
|---|---|---|
| Development Team | Software developers responsible for building the system. | Design, implement, test, deploy, and maintain the application. |
| Project Supervisor | Academic supervisor guiding the project. | Review progress, provide feedback, and evaluate project quality. |

---

## Stakeholder Requirements

### Patient Requirements

Patients expect the system to:

- Provide a secure registration and login process.
- Allow searching for doctors using multiple criteria.
- Display detailed doctor profiles.
- Support online appointment booking.
- Allow appointment cancellation.
- Display appointment history.
- Enable profile management.
- Allow submitting reviews after completed appointments.

---

### Doctor Requirements

Doctors expect the system to:

- Maintain a professional profile.
- Create and manage weekly schedules.
- Receive appointment requests.
- Accept or reject appointment requests.
- Complete appointments.
- Access patient information.
- View appointment statistics.

---

### Administrator Requirements

Administrators require the ability to:

- Monitor system activity.
- Manage doctor accounts.
- Activate and deactivate doctors.

- Manage clinics.
- Manage medical specializations.
- View registered patients.
- Monitor all appointments.
- Access statistical reports.

---

### Stakeholder Relationship

The interaction between stakeholders can be summarized as follows:

- **Patients** interact primarily with doctors through appointment booking and reviews.
- **Doctors** interact with patients by managing appointments and schedules.
- **Administrators** oversee both patients and doctors while managing system resources such as clinics and specializations.
- **Development Team** implements and maintains the system based on stakeholder requirements.
- **Project Supervisor** monitors project quality and ensures adherence to software engineering standards.

---

## 3.2 User Stories & Use Cases

### User Stories

The following user stories describe the main functionalities expected by each type of user.

---

### Patient User Stories

| ID | User Story |
|---|---|
| US-01 | As a patient, I want to register an account so that I can access the system. |
| US-02 | As a patient, I want to log in securely so that I can manage my appointments. |
| US-03 | As a patient, I want to update my profile information. |
| US-04 | As a patient, I want to browse available doctors. |
| US-05 | As a patient, I want to search for doctors by name or specialization. |

| | |
|---|---|
| US-06 | As a patient, I want to filter doctors by specialization, location, and gender. |
| US-07 | As a patient, I want to view doctor profiles before booking. |
| US-08 | As a patient, I want to book an available appointment. |
| US-09 | As a patient, I want to cancel my appointment if necessary. |
| US-10 | As a patient, I want to view my upcoming and previous appointments. |
| US-11 | As a patient, I want to submit a review after my appointment is completed. |

---

## Doctor User Stories

| ID | User Story |
|---|---|
| US-12 | As a doctor, I want to manage my profile information. |
| US-13 | As a doctor, I want to create my weekly schedule. |
| US-14 | As a doctor, I want to edit or delete my schedules. |
| US-15 | As a doctor, I want to accept or reject appointment requests. |
| US-16 | As a doctor, I want to mark appointments as completed. |
| US-17 | As a doctor, I want to view patient information before appointments. |
| US-18 | As a doctor, I want to monitor appointment statistics. |

---

## Administrator User Stories

| ID | User Story |
|---|---|
| US-19 | As an administrator, I want to manage doctor accounts. |
| US-20 | As an administrator, I want to activate or deactivate doctors. |
| US-21 | As an administrator, I want to manage clinics. |
| US-22 | As an administrator, I want to manage medical specializations. |
| US-23 | As an administrator, I want to monitor appointments. |
| US-24 | As an administrator, I want to view dashboard statistics. |

## Use Cases

The main use cases of the system include:

| Use Case ID | Use Case | Primary Actor |
|---|---|---|
| UC-01 | Register Account | Patient |
| UC-02 | Login | All Users |
| UC-03 | Change Password | All Users |
| UC-04 | Manage Patient Profile | Patient |
| UC-05 | Browse Doctors | Patient |
| UC-06 | Search & Filter Doctors | Patient |
| UC-07 | View Doctor Profile | Patient |
| UC-08 | Book Appointment | Patient |
| UC-09 | Cancel Appointment | Patient |
| UC-10 | View Appointment History | Patient |
| UC-11 | Submit Review | Patient |
| UC-12 | Manage Doctor Profile | Doctor |
| UC-13 | Manage Schedule | Doctor |
| UC-14 | Manage Appointments | Doctor |
| UC-15 | View Patient Information | Doctor |
| UC-16 | View Statistics | Doctor |
| UC-17 | Manage Doctors | Administrator |
| UC-18 | Manage Clinics | Administrator |
| UC-19 | Manage Specializations | Administrator |
| UC-20 | Monitor Appointments | Administrator |

**Note:** Detailed Use Case Diagrams and specifications are provided in **Section 4 – System Analysis & Design**.

---

# 3.3 Functional Requirements

The functional requirements define the operations and services that the Doktorak system must provide to its users.

### Authentication & User Management

- User Registration
- User Login
- Change Password
- Role-Based Access Control

---

### Patient Profile Management

- View Patient Profile
- Edit Patient Profile
- Upload Profile Picture

---

### Doctor Discovery

- Browse Doctors
- Search Doctors
- Filter by Specialization
- Filter by Location
- Filter by Gender
- Sort Doctors
- View Doctor Profile

---

### Appointment Management (Patient)

- Book Appointment
- Cancel Appointment
- View Upcoming Appointments

- View Appointment History
- Appointment Status Management

---

## Feedback & Reviews

- Submit Doctor Reviews
- View Doctor Reviews

---

## Doctor Profile Management

- View Doctor Profile
- Edit Doctor Profile
- Upload Doctor Profile Picture

---

## Schedule Management

- Create Schedule
- Update Schedule
- Delete Schedule

---

## Doctor Appointment Management

- View Assigned Appointments
- Accept Appointment
- Reject Appointment
- Complete Appointment

---

## Patient Information

- View Patient Profile

---

## Statistics

- View Appointment Statistics

---

### Administrator Functions

- View Dashboard
- View Monthly Statistics (Optional)
- Add Doctor
- Activate Doctor
- Deactivate Doctor
- View Patients
- Manage Specializations (CRUD)
- View All Appointments
- Search Appointments
- Add Clinic
- Edit Clinic
- Delete Clinic

---

## 3.4 Non-functional Requirements

Non-functional requirements describe the quality attributes and operational constraints of the Doktorak system.

### Performance Requirements

- The system should respond to user requests within **3 seconds** under normal operating conditions.
- Doctor search and appointment retrieval should be completed efficiently.
- The system should support multiple concurrent users without significant performance degradation.

---

### Security Requirements

- All users must authenticate before accessing protected resources.
- Passwords shall be securely hashed before storage.
- Role-Based Access Control (RBAC) shall restrict access according to user roles.
- User inputs shall be validated to prevent common security vulnerabilities.
- Unauthorized users shall not access restricted modules.

---

## Reliability Requirements

- The system should provide consistent and accurate appointment management.
- Appointment records shall be preserved without data loss.
- Database transactions shall maintain data consistency.

---

## Availability Requirements

- The system should be available to users at all times except during scheduled maintenance.
- Users should be able to access the application through any modern web browser with an internet connection.

---

## Usability Requirements

- The user interface should be intuitive and easy to navigate.
- Consistent layouts and responsive design should improve user experience across different devices.
- Error messages should clearly explain validation or system issues.

---

## Maintainability Requirements

- The application shall follow a modular architecture to simplify maintenance.
- Source code should follow coding standards and naming conventions.
- Documentation should be maintained to support future development.

---

## Scalability Requirements

- The system should support future expansion, including additional healthcare services and increased numbers of users.
- New modules should be integrated with minimal changes to existing components.

---

## Compatibility Requirements

- The system shall support major web browsers such as Google Chrome, Microsoft Edge, Mozilla Firefox, and Safari.
- The application shall be responsive on desktop, tablet, and mobile devices.

---

### Data Integrity Requirements

- Each appointment shall have a valid patient, doctor, and appointment status.
- Duplicate appointment bookings for the same doctor and time slot shall not be allowed.
- Data entered into the system shall comply with defined validation rules.

---

### Backup and Recovery Requirements

- The database should be backed up periodically to prevent data loss.
- Recovery procedures should be available in the event of hardware or software failure.

---

# 4. System Analysis & Design

# 4.1 Use Case Diagram

The system has three primary actors:

- Patient
- Doctor
- Administrator

# Use Case Descriptions

Since the complete SRS already contains the detailed functional requirements (FR-1 to FR-45), this section summarizes the major use cases.

---

## UC-01 Register

| Item | Description |
|---|---|
| Primary Actor | Patient |
| Description | Creates a new patient account. |
| Preconditions | User is not registered. |
| Main Flow | User enters information → System validates → Account created. |
| Postconditions | Patient account is created successfully. |

---

## UC-02 Login

| Item | Description |
|---|---|
| Primary Actor | Patient / Doctor / Administrator |
| Description | Authenticates registered users. |
| Preconditions | Valid account exists. |
| Main Flow | User enters credentials → System verifies → Dashboard displayed. |
| Postconditions | Authenticated session created. |

---

## UC-03 Book Appointment

| Item | Description |
|---|---|

| | |
|---|---|
| Primary Actor | Patient |
| Description | Books an available appointment with a doctor. |
| Preconditions | User logged in and selected slot available. |
| Main Flow | Select doctor → Select date → Select slot → Confirm booking. |
| Postconditions | Appointment status becomes **Pending**. |

---

## UC-04 Manage Schedule

| Item | Description |
|---|---|
| Primary Actor | Doctor |
| Description | Create, edit, and delete available schedules. |
| Preconditions | Doctor logged in. |
| Main Flow | Enter schedule information → Validate → Save. |
| Postconditions | Schedule updated successfully. |

---

## UC-05 Manage Doctors

| Item | Description |
|---|---|
| Primary Actor | Administrator |
| Description | Add, activate, or deactivate doctor accounts. |
| Preconditions | Administrator logged in. |
| Main Flow | Select action → Validate → Save changes. |
| Postconditions | Doctor information updated. |

---

## UC-06 Manage Clinics

| Item | Description |
|---|---|
| Primary Actor | Administrator |
| Description | Create, update, and delete clinics. |
| Preconditions | Administrator logged in. |
| Postconditions | Clinic information updated. |

---

## UC-07 Manage Specializations

| Item | Description |
|---|---|
| Primary Actor | Administrator |
| Description | Create, edit, and delete medical specializations. |
| Preconditions | Administrator logged in. |
| Postconditions | Specialization list updated. |

# 4.2 Software Architecture

## Architectural Style

Doktorak follows the **ASP.NET Core MVC (Model–View–Controller)** architectural pattern.

The architecture separates the application into independent components to improve maintainability, scalability, and code organization.

The system consists of four logical layers:

1. Presentation Layer
2. Business Logic Layer

3. Data Access Layer
4. Database Layer

This layered architecture promotes **Separation of Concerns**, making each layer responsible for a specific functionality.

---

# Architecture Overview

## Presentation Layer

Responsible for user interaction.

Components include:

- MVC Controllers
- Razor Views
- HTML
- CSS
- Bootstrap
- JavaScript

Responsibilities:

- Receive HTTP requests.
- Display data.
- Validate client-side input.
- Send user actions to the Business Layer.

---

## Business Logic Layer

Responsible for implementing business rules.

Responsibilities:

- Appointment validation.
- Booking rules.
- Authentication.
- Authorization.
- Schedule generation.
- Profile management.
- Review validation.

---

## Data Access Layer

Responsible for communication with SQL Server.

Components:

- Entity Framework Core
- DbContext
- Repositories

Responsibilities:

- CRUD operations.
- Database queries.
- Data persistence.
- Transaction management.

---

## Database Layer

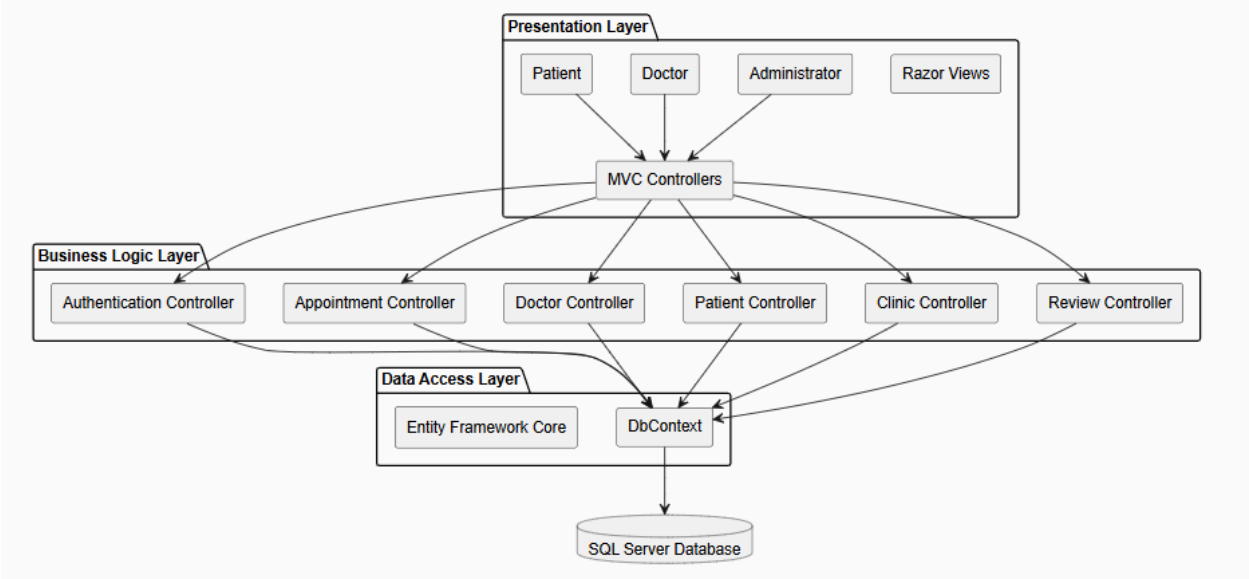
Stores all application data.

Main entities include:

- Users
- Patients
- Doctors
- Clinics
- Specializations
- Appointments
- Schedules
- Reviews

---

# Architecture Diagram

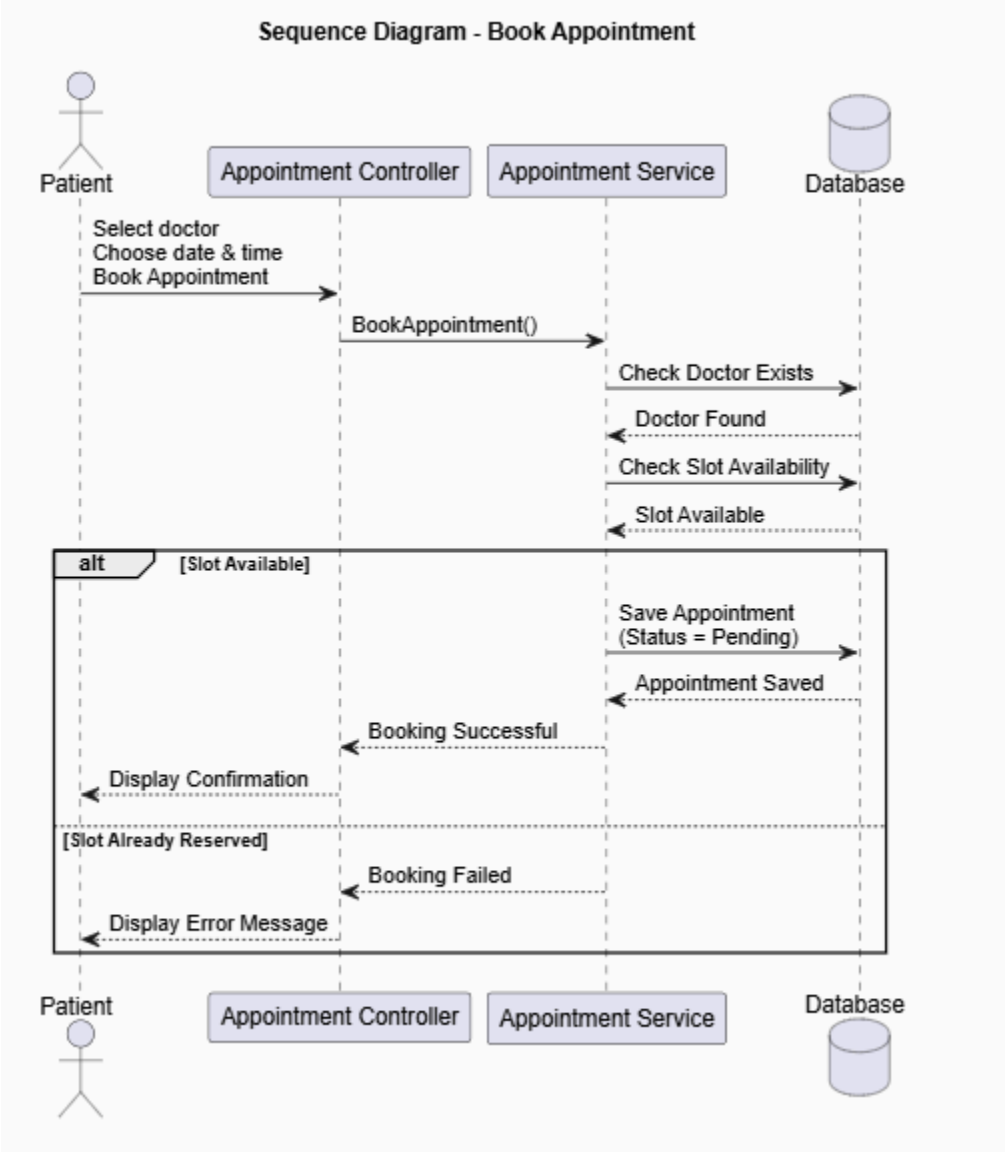


# 4.3 Database Design & Data Modeling

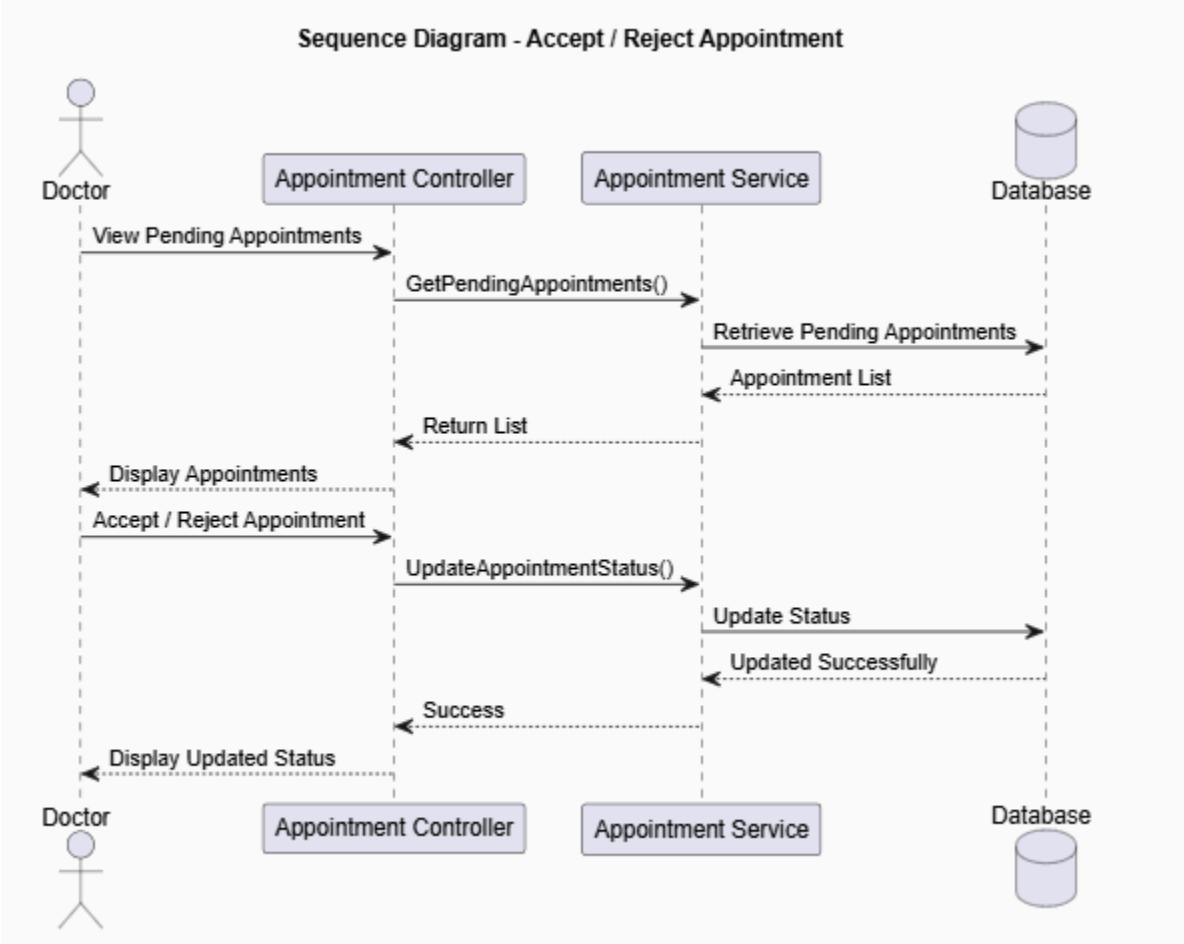
## ERD

# 4.4 Sequence Diagrams

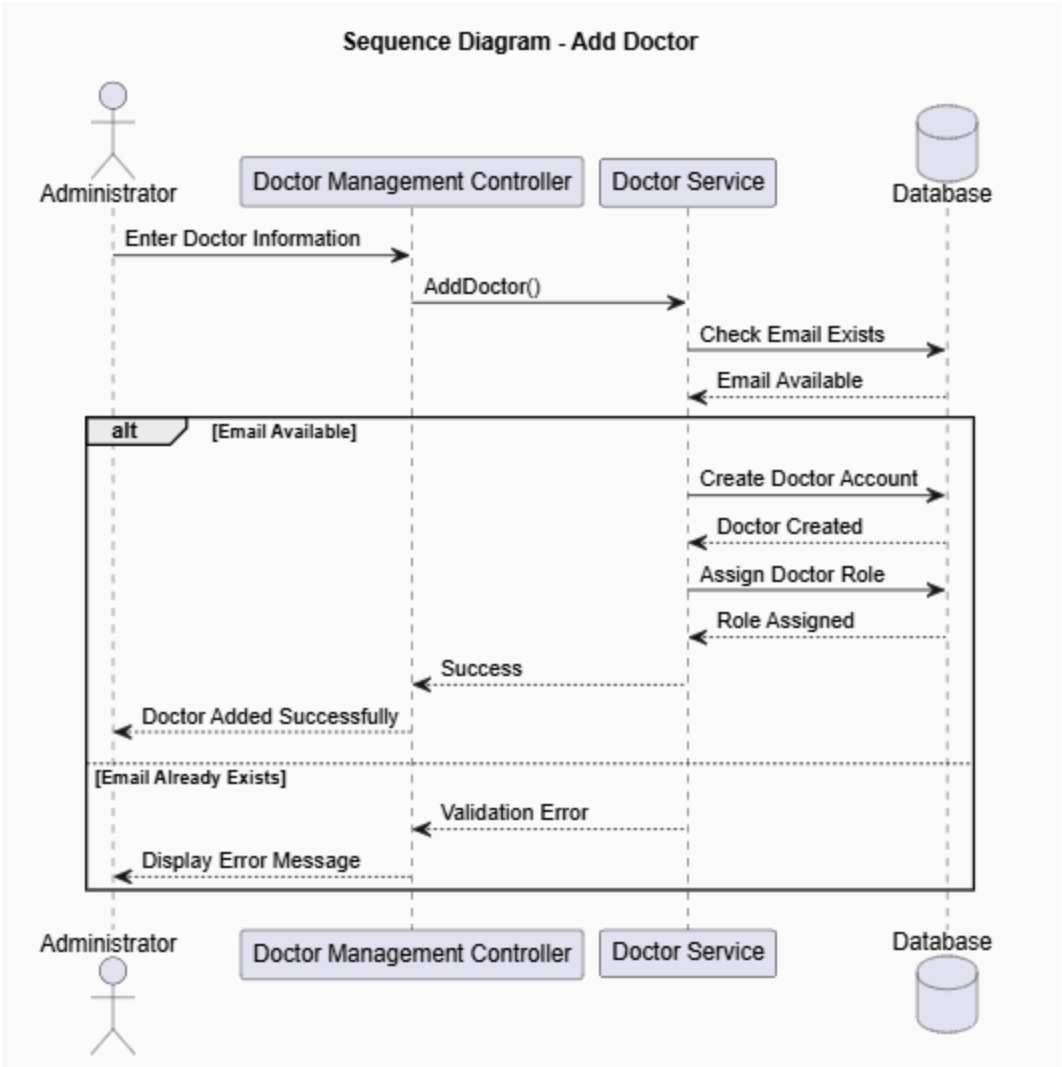
## 1. Sequence Diagram – Book Appointment



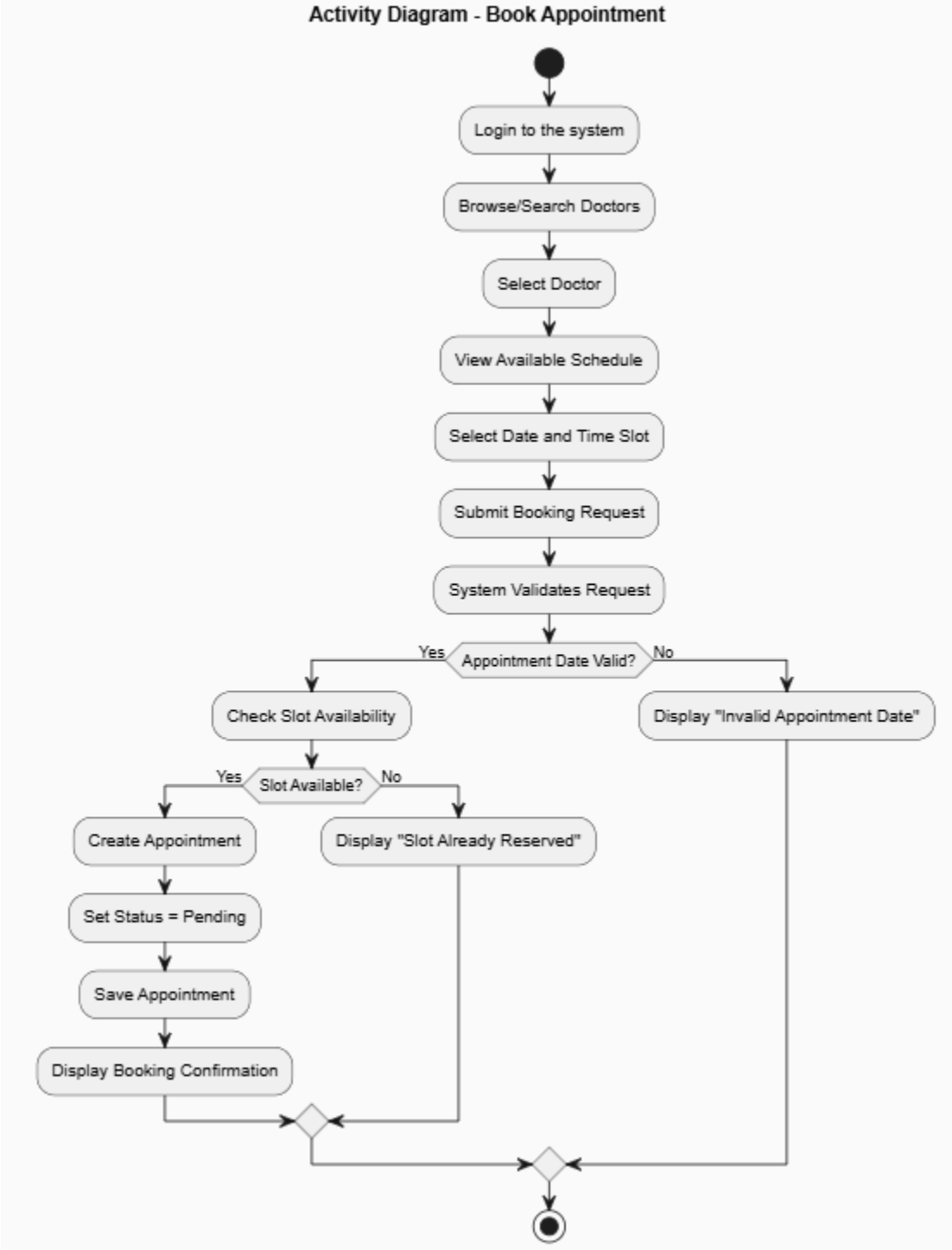
## 2. Sequence Diagram – Doctor Accepts or Rejects Appointment



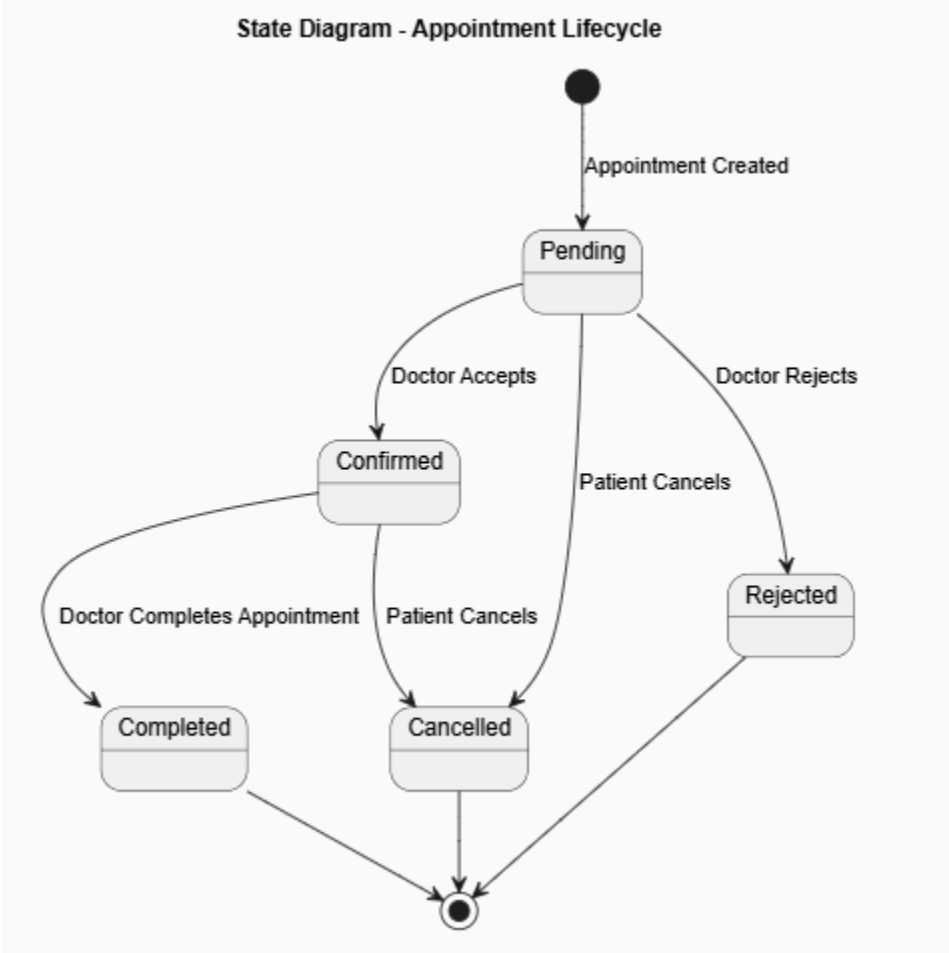

## 3. Sequence Diagram – Administrator Adds a Doctor


Sequence Diagram - Add Doctor
Administrator
Doctor Management Controller
Doctor Service
Database
Enter Doctor Information
AddDoctor()
Check Email Exists
Email Available
alt
[Email Available]
Create Doctor Account
Doctor Created
Assign Doctor Role
Role Assigned
Success
Doctor Added Successfully
[Email Already Exists]
Validation Error
Display Error Message
Administrator
Doctor Management Controller
Doctor Service
Database

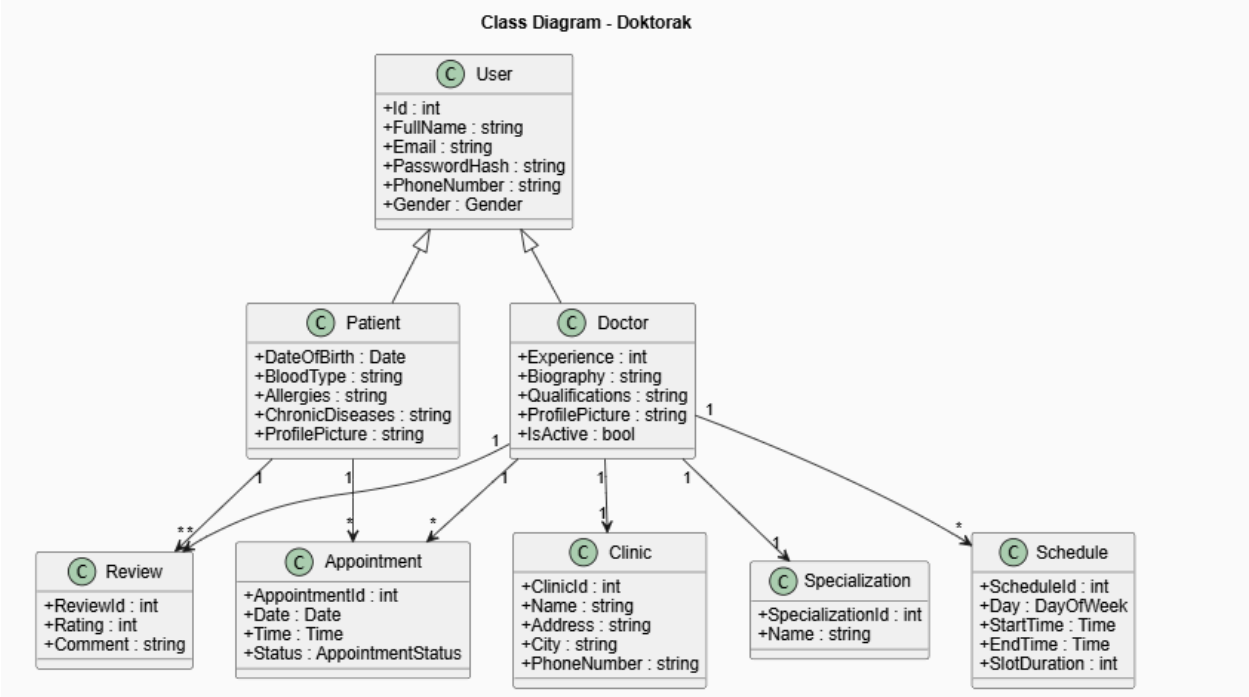
# 4.5 Activity Diagram



# 4.6 State Diagram



# 4.7 Class Diagram




---

# 4.8 UI/UX Design & Prototyping

## Introduction

The user interface of **Doktorak** was designed with a strong emphasis on simplicity, accessibility, and usability. Since the system is intended for patients, doctors, and administrators with varying levels of technical expertise, the interfaces were designed to be intuitive, responsive, and consistent across all modules.

The design follows modern web application principles by providing clear navigation, consistent layouts, readable typography, and responsive components that adapt to different screen sizes.

---

# 4.8.1 Wireframes & Mockups

### Overview

Before implementation, wireframes and interface mockups were created to visualize the layout and functionality of each page. These prototypes helped identify user requirements, improve navigation flow, and ensure consistency throughout the application.

The final implementation closely follows these wireframes while incorporating responsive design principles using Bootstrap.

---

# Login Page

### Purpose

Provides secure authentication for Patients, Doctors, and Administrators.

### Main Components

- Email field
- Password field
- Login button
- Forgot Password link (if implemented)
- Register link for patients
- Validation messages

### Design Considerations

- Minimal and clean layout.
- Centered login form.
- Immediate validation feedback.
- Clear call-to-action buttons.

---

# Registration Page

### Purpose

Allows new patients to create an account.

### Main Components

- Full Name
- Email Address
- Password
- Confirm Password
- Phone Number
- Gender
- Date of Birth
- Register Button

### Design Considerations

- Multi-field form organized vertically.
- Required fields clearly indicated.
- Password strength validation.
- Consistent spacing between controls.

---

# Patient Dashboard

### Purpose

Provides patients with quick access to system features.

### Main Components

- Navigation menu
- Doctor search shortcut
- Upcoming appointments
- Appointment history
- Profile shortcut

### Design Considerations

- Dashboard cards for quick navigation.
- Minimal clicks to access frequently used features.
- Responsive layout.

---

# Doctors Page

### Purpose

Allows patients to browse and search available doctors.

### Main Components

- Search bar
- Filter panel
- Doctor cards
- Sorting options

Each doctor card contains:

- Profile picture
- Name
- Specialization
- Clinic
- Experience
- Rating
- View Profile button

### Design Considerations

- Card-based layout.
- Easy filtering.
- Responsive grid.
- Large profile images.

---

## Doctor Profile Page

### Purpose

Displays detailed information about a selected doctor.

### Main Components

- Doctor photo
- Personal information
- Qualifications
- Biography
- Clinic information
- Available appointment slots
- Patient reviews
- Book Appointment button

### Design Considerations

- Information grouped into sections.
- Available slots highlighted.
- Reviews displayed below profile details.

---

# Appointment Booking Page

### Purpose

Allows patients to reserve appointments.

### Main Components

- Doctor information
- Calendar
- Available slots
- Confirmation button

### Design Considerations

- Simple booking workflow.
- Prevent unavailable slots from being selected.
- Confirmation message after booking.

---

# Doctor Dashboard

### Purpose

Provides doctors with tools to manage appointments and schedules.

### Main Components

- Appointment statistics
- Today's appointments
- Schedule management
- Profile management

### Design Considerations

- Dashboard widgets.
- Quick access to pending appointments.
- Clear appointment status indicators.

---

# Administrator Dashboard

## Purpose

Provides administrators with complete system management.

## Main Components

- Statistics cards
- Doctor Management
- Patient Management
- Clinic Management
- Specialization Management
- Appointment Monitoring

## Design Considerations

- Administrative functions organized by modules.
- Dashboard cards for system statistics.
- Simple navigation menu.

---

# Responsive Design

The interfaces were designed to function correctly across different devices.

Supported screen sizes include:

- Desktop computers
- Laptops
- Tablets
- Mobile phones

Bootstrap's responsive grid system was used to automatically adapt layouts based on screen resolution.

---

# 4.8.2 UI/UX Guidelines

The following design guidelines were followed throughout the development of the Doktorak system.

---

## Consistency

All pages use a consistent visual style, including:

- Common navigation structure
- Uniform button styles
- Consistent typography
- Standardized color scheme
- Reusable UI components

This consistency reduces the learning curve for users and improves usability.

---

## Simplicity

Interfaces were designed to minimize unnecessary complexity.

The system avoids excessive visual elements and focuses on presenting only the information required for completing user tasks efficiently.

---

## Navigation

Navigation was designed to be intuitive.

Each user role has access only to the features relevant to their responsibilities.

Patients, doctors, and administrators each have dedicated dashboards and navigation menus.

---

## Readability

To improve readability:

- Clear section headings are used.
- Adequate spacing is maintained.
- Font sizes are consistent.
- Labels are descriptive.
- Forms are properly aligned.

---

## Accessibility

The interface follows accessibility best practices by:

- Using readable font sizes.
- Providing sufficient color contrast.
- Displaying descriptive validation messages.
- Using clearly labeled form controls.
- Supporting keyboard navigation where applicable.

---

## Feedback

The system provides immediate feedback for user actions.

Examples include:

- Successful login messages.
- Appointment booking confirmations.
- Validation error messages.
- Success notifications after profile updates.
- Error messages when operations fail.

This helps users understand the result of their actions.

---

## Error Prevention

To reduce user errors, the system includes:

- Required field validation.
- Password complexity validation.

- Email uniqueness validation.
- Phone number validation.
- Appointment date validation.
- Prevention of duplicate bookings.
- Prevention of selecting unavailable appointment slots.

---

## Responsive Design Principles

The UI adapts automatically to different screen sizes by using Bootstrap's responsive grid system.

Key responsive features include:

- Flexible layouts.
- Responsive navigation menus.
- Mobile-friendly forms.
- Responsive tables.
- Adaptive cards and images.

---

## Color Scheme

The Doktorak interface follows a healthcare-inspired color palette to provide a professional and trustworthy user experience.

Recommended colors include:

| Color | Usage |
|---|---|
| Blue | Primary buttons, navigation bar, headers |
| Green | Success messages, confirmed appointments |
| Red | Validation errors, rejected or cancelled appointments |
| Gray | Secondary buttons, borders, background elements |
| White | Page backgrounds and content areas |

---

# Typography

The interface uses clean and modern typography to improve readability.

Typography guidelines include:

- Sans-serif fonts for readability.
- Consistent font sizes across pages.
- Bold headings for section titles.
- Clear distinction between labels and input fields.

---

# Icons

Meaningful icons are used throughout the application to improve usability.

Examples include:

| Icon | Purpose |
| --- | --- |
| 👤 | User Profile |
| 🩺 | Doctors |
| 📅 | Appointments |
| 🏥 | Clinics |
| ⭐ | Reviews |
| 📊 | Dashboard<br>Statistics |
| ⚙️ | Settings |

---

# User Experience Goals

The UI/UX design aims to achieve the following objectives:

- Minimize the number of steps required to complete common tasks.
- Provide a consistent experience across all user roles.
- Reduce user errors through validation and clear feedback.

- Ensure fast navigation between modules.
- Support users with different levels of technical expertise.
- Deliver a responsive and visually appealing interface across all devices.

---

# 4.9 System Deployment & Integration

## Introduction

System deployment and integration describe the technologies, software components, and deployment environment used to build and run the **Doktorak Doctor Appointment Management System**. The system follows a layered ASP.NET Core MVC architecture and integrates the presentation layer, business logic, data access layer, and SQL Server database into a single web application.

---

# 4.9.1 Technology Stack

The Doktorak system was developed using modern web development technologies to ensure scalability, maintainability, and performance.

| Layer | Technology | Purpose |
|---|---|---|
| Programming Language | C# | Backend development |
| Framework | ASP.NET Core MVC | Web application framework |
| ORM | Entity Framework Core | Database access and object-relational mapping |
| Database | Microsoft SQL Server | Data storage |
| Frontend | HTML5 | Page structure |
| Styling | CSS3 | User interface styling |
| UI Framework | Bootstrap 5 | Responsive design |

| | | |
|---|---|---|
| Client-side Scripting | JavaScript | Client-side interactivity |
| Authentication | ASP.NET Core Identity | User authentication and authorization |
| IDE | Visual Studio 2022 | Development environment |
| Database Tool | SQL Server Management Studio (SSMS) | Database management |
| Version Control | Git & GitHub | Source code management |
| Hosting | MonsterASP.NET | Application deployment |

---

# Technology Description

## ASP.NET Core MVC

ASP.NET Core MVC was selected as the primary web framework because it provides:

- Separation of Concerns
- High performance
- Built-in Dependency Injection
- Strong security features
- Cross-platform support
- Easy maintenance

---

## Entity Framework Core

Entity Framework Core simplifies communication between the application and SQL Server by providing:

- Object Relational Mapping (ORM)
- LINQ support
- CRUD operations
- Migration support
- Change tracking

---

## SQL Server

SQL Server stores all application data including:

- Users
- Patients
- Doctors
- Clinics
- Specializations
- Schedules
- Appointments
- Reviews

It ensures data consistency, reliability, and security.

---

## Bootstrap 5

Bootstrap was used to create a responsive and modern user interface.

Advantages include:

- Responsive grid system
- Ready-made components
- Cross-browser compatibility
- Mobile-first design

---

## Git & GitHub

GitHub was used for:

- Version control
- Team collaboration
- Branch management
- Pull requests
- Code review

---

## MonsterASP.NET

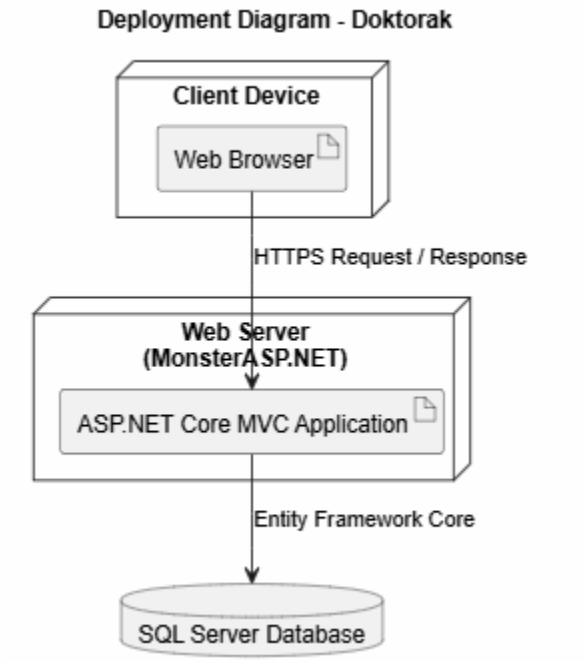
MonsterASP.NET was selected as the hosting provider because it supports:

- ASP.NET Core applications
- SQL Server databases

- Web Deploy publishing
- HTTPS support
- Easy deployment from Visual Studio

---

# 4.9.2 Deployment Diagram




## Deployment Description

The deployment consists of three primary nodes:

### Client Device

Represents the user's computer, tablet, or smartphone running a web browser.

Users include:

- Patients
- Doctors
- Administrators

---

## Web Server

Hosts the ASP.NET Core MVC application and is responsible for:

- Processing HTTP requests
- Performing authentication and authorization
- Executing business logic
- Communicating with the database
- Returning HTML responses to clients

---

## Database Server

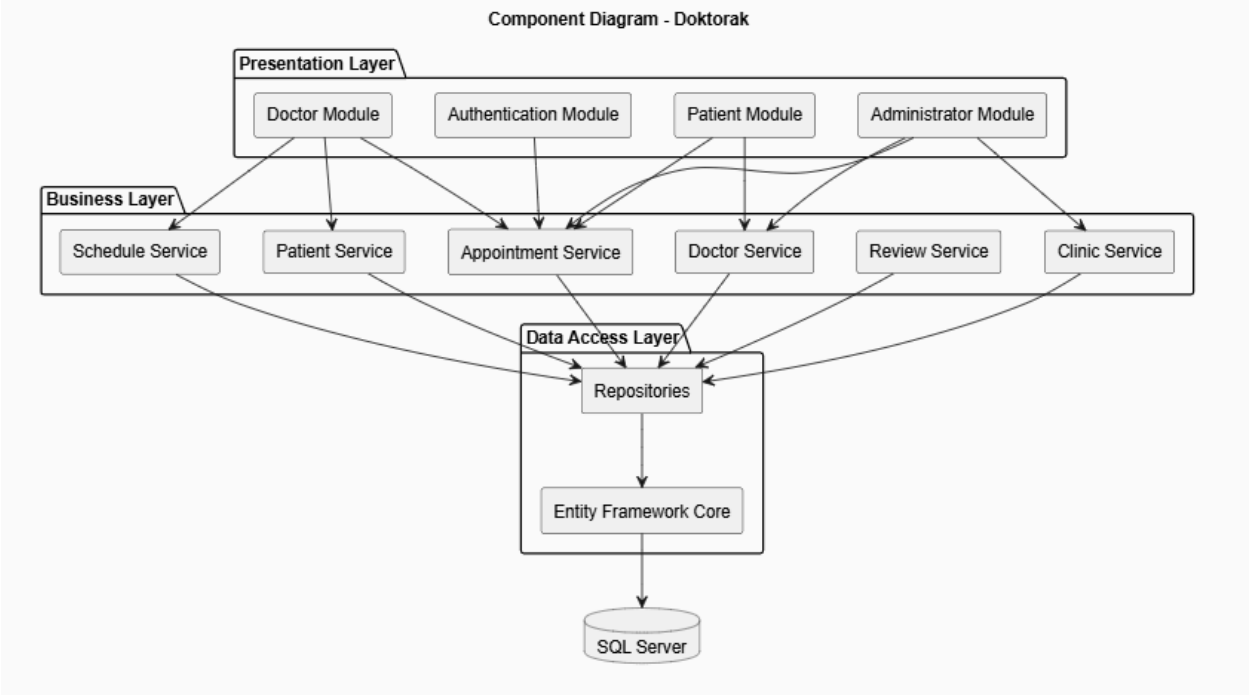
Stores all persistent application data.

Responsibilities include:

- Managing user accounts
- Storing doctor and patient information
- Managing appointments
- Maintaining schedules
- Storing clinics, specializations, and reviews

---

# 4.9.3 Component Diagram




## Component Description

### Authentication Module

Responsible for:

- User registration
- User login
- Password management
- Role-based authorization

---

### Patient Module

Provides functionality for patients, including:

- Profile management
- Doctor browsing
- Appointment booking
- Appointment cancellation
- Review submission

---

## Doctor Module

Provides functionality for doctors, including:

- Profile management
- Schedule management
- Appointment management
- Viewing patient information
- Appointment statistics

---

## Administrator Module

Provides administrative functionality, including:

- Doctor management
- Clinic management
- Specialization management
- Appointment monitoring
- Dashboard statistics

---

## Business Services

The business layer contains services responsible for implementing application logic:

- Appointment Service
- Doctor Service
- Patient Service
- Clinic Service
- Review Service
- Schedule Service

These services validate requests, enforce business rules, and coordinate data access operations.

---

## Data Access Layer

The data access layer consists of:

- Entity Framework Core
- Repository classes

Its responsibilities include:

- Retrieving data from SQL Server
- Performing CRUD operations
- Managing database transactions
- Persisting application data

---

### SQL Server Database

The database stores all application entities, including:

- Users
- Patients
- Doctors
- Clinics
- Specializations
- Schedules
- Appointments
- Reviews

It serves as the central repository for all system data and ensures data consistency and integrity.

---

# 4.10 Testing & Validation

## Introduction

Testing was performed throughout the development lifecycle to ensure that all functional requirements were implemented correctly and that the system behaved as expected under different scenarios.

The testing process focused primarily on **functional testing**, validating each feature from the perspective of patients, doctors, and administrators.

---

# Testing Approach

The following testing methods were applied:

- Functional Testing
- Integration Testing
- User Interface Testing
- Validation Testing
- Database Testing

---

# Functional Testing

The following table summarizes representative functional test cases.

| Test Case | Expected Result | Status |
|---|---|---|
| Patient Registration | New patient account created successfully | Passed |
| User Login | User redirected to the appropriate dashboard | Passed |
| Doctor Search | Matching doctors displayed correctly | Passed |
| Filter Doctors | Doctors filtered according to selected criteria | Passed |
| Book Appointment | Appointment created with **Pending** status | Passed |
| Cancel Appointment | Appointment status updated to **Cancelled** | Passed |
| Accept Appointment | Status updated to **Confirmed** | Passed |
| Reject Appointment | Status updated to **Rejected** | Passed |
| Complete Appointment | Status updated to **Completed** | Passed |

| | | |
|---|---|---|
| Submit Review | Review stored successfully | Passed |
| Add Doctor | Doctor account created successfully | Passed |
| Manage Clinics | CRUD operations completed successfully | Passed |
| Manage Specializations | CRUD operations completed successfully | Passed |

---

## Validation Testing

Input validation was implemented throughout the application to maintain data integrity.

Examples include:

- Required field validation.
- Email format validation.
- Unique email validation.
- Password complexity validation.
- Phone number validation.
- Appointment date validation.
- Prevention of duplicate bookings.
- Prevention of booking unavailable appointment slots.

---

# Database Testing

Database testing verified:

- Correct insertion of records.
- Update operations.
- Delete operations.
- Relationship consistency.
- Foreign key integrity.
- Data retrieval accuracy.

---

## User Interface Testing

The user interface was tested to verify:

- Correct page navigation.
- Responsive layouts.
- Proper display on different screen sizes.
- Validation message visibility.
- Correct button behavior.
- Consistent styling across pages.

---

## Test Results Summary

The testing process confirmed that:

- All core functional requirements operate correctly.
- User authentication functions as expected.
- Appointment management follows the defined workflow.
- Administrative operations execute successfully.
- Input validation prevents invalid data entry.
- Database operations maintain consistency and integrity.

Overall, the system met its functional objectives and demonstrated stable performance during testing.

---

# 4.11 Deployment Strategy

## Deployment Overview

The Doktorak application was deployed as an **ASP.NET Core MVC** web application using **MonsterASP.NET** hosting services.

The deployment process included publishing the application from Visual Studio and connecting it to a Microsoft SQL Server database hosted on the same platform.

---

# Deployment Environment

| Component | Technology |
|---|---|
| Hosting Provider | MonsterASP.NET |
| Application Framework | ASP.NET Core MVC |
| Database | Microsoft SQL Server |
| Deployment Method | Visual Studio Web Deploy |
| Version Control | GitHub |

---

# Deployment Steps

The deployment process consisted of the following steps:

1. Complete development and testing.
2. Push the latest source code to the GitHub repository.
3. Publish the project in **Release** mode.
4. Import the **MonsterASP.NET Publish Profile** into Visual Studio.
5. Publish the application using **Web Deploy**.
6. Configure the production database connection string.
7. Verify successful deployment.
8. Perform final functional testing on the live application.

---

# Deployment Architecture

The deployment environment consists of:

- Client devices (web browsers).
- MonsterASP.NET web server hosting the ASP.NET Core MVC application.
- SQL Server database storing application data.

Users communicate with the application through HTTPS requests, while the application accesses the database through Entity Framework Core.

---

# Version Control Strategy

GitHub was used throughout the project to support collaborative development.

The team followed a branch-based workflow:

- Developers implemented features in separate branches.
- Changes were committed with descriptive commit messages.
- Pull Requests were reviewed before merging into the main branch.
- The main branch always contained the latest stable version of the project.

---

## Maintenance Strategy

Future maintenance activities include:

- Monitoring application availability.
- Applying security updates.
- Fixing reported bugs.
- Optimizing database performance.
- Enhancing user experience.
- Implementing new features based on user feedback.

---

## Backup and Recovery

To ensure data reliability:

- Source code is backed up using GitHub.
- SQL Server database backups should be performed periodically.
- Recovery procedures should be followed in the event of deployment or database failures.